

Lab - Running Our Application Container

To Begin with the Lab

Summary

This lab demonstrates how to build a custom Docker image from a Dockerfile and run your application as a Docker container. The Docker build process reads the Dockerfile step by step, builds the application, creates the image, and prepares it for execution.

Understanding

A Docker image is created by executing the instructions inside the **Dockerfile**. The docker build command reads the Dockerfile, performs each step in sequence, and packages the application into an image. During the build, Docker may use multiple stages, such as a build stage to compile the application and a runtime stage to create a smaller production image. The image is identified by a name and an optional version tag, making it easy to manage different versions.

Example:

```
docker build -t dockerapp:v1 .
```

- `docker build` → Builds a Docker image.
- `-t dockerapp:v1` → Tags the image with the name **dockerapp** and version **v1**.
- `.` → Uses the current directory as the build context, including the Dockerfile.

Lab Steps

Step 1: Open the Project Folder

- Open the terminal.
- Navigate to the folder containing the application.
- Ensure the **Dockerfile** is present in the current directory.



Step 2: Build the Docker Image

- Run the Docker build command.
- Assign an image name.
- Assign a version tag.

```
docker build -t dockerapp:v1 .
```

```
PS D:\code\DockerApp> docker build -t dockerapp:v1 .
[+] Building 140.4s (15/15) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 581B
=> [internal] load metadata for mcr.microsoft.com/dotnet/sdk:10.0
=> [internal] load metadata for mcr.microsoft.com/dotnet/aspnet:10.0
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [build 1/6] FROM mcr.microsoft.com/dotnet/sdk:10.0@sha256:ed034a8bf0b24ded0cbbac07e17825d8e9ebfe21e308191d0f7421eaf5a 123.7s
=> => resolve mcr.microsoft.com/dotnet/sdk:10.0@sha256:ed034a8bf0b24ded0cbbac07e17825d8e9ebfe21e308191d0f7421eaf5a 0.0s
=> => sha256:6973830c09c3da5aeabb0c5638bc3617879816fd70de6af2625449f467f8ec0 18.05MB / 18.05MB 17.3s
=> => sha256:8527f9bfc4711c48d4dbc4c10ce21337bc93c0a305a5f9e116f754f5754ecbb0 496B / 496B 1.0s
=> => sha256:d4555a6a451011811d13a9b24114d2a04d3a3afb7d428597812ff4bf18c8e45 185.25MB / 185.25MB 97.0s
=> => sha256:5cece633748fe30407f0c3bb9b383840cdf5de70801f8293ce2ed7d62f24c443 26.24MB / 26.24MB 47.0s
=> => extracting sha256:5cece633748fe30407f0c3bb9b383840cdf5de70801f8293ce2ed7d62f24c443 0.6s
=> => extracting sha256:d4555a6a451011811d13a9b24114d2a04d3a3afb7d428597812ff4bf18c8e45 2.5s
=> => extracting sha256:8527f9bfc4711c48d4dbc4c10ce21337bc93c0a305a5f9e116f754f5754ecbb0 0.0s
=> => extracting sha256:6973830c09c3da5aeabb0c5638bc3617879816fd70de6af2625449f467f8ec0 0.3s
=> [runtime 1/3] FROM mcr.microsoft.com/dotnet/aspnet:10.0@sha256:1fa23fc4872d95fd71c2833ebe65d7e84a43b2d51a31d119516852f 66.5s
=> => resolve mcr.microsoft.com/dotnet/aspnet:10.0@sha256:1fa23fc4872d95fd71c2833ebe65d7e84a43b2d51a31d119516852f 0.0s
=> => sha256:2579678c60073aab2b43b6cd005500226891c1dc09017f813d272868855c93 12.73MB / 12.73MB 5.8s
=> => sha256:0287f08f0ac044b026b08b63a9ce1f57fffd69363f315a0c30a3dbf61b70efc2 155B / 155B 0.7s
=> => sha256:8e782b788f5dc9751512787f6ef833c7a7a3fbf397653ef3fb09a59f2224236 36.60MB / 36.60MB 56.8s
=> => sha256:7090f4cc64dfe013b79ca3453fb62301f7a9c635d0e4a06e6a9440f794e0c421 3.57kB / 3.57kB 0.8s
=> => sha256:edf564afb68fae053e8536d2b24e46f40090b6653def00f8df93cc708160f699 16.78MB / 16.78MB 23.0s
=> => sha256:ca2628b2709c15185702954d211b1a6406195114bf755f58b6c25d32b3888 29.74MB / 29.74MB 63.8s
```

Step 3: Understand the Build Context

- The represents the current directory.
- Docker sends all required files from the current folder.
- Docker locates the Dockerfile automatically.

Step 4: Read the Dockerfile

- Docker reads the Dockerfile.
- Instructions are executed one by one.
- Each instruction creates a new image layer.

Step 5: Build the Application

- Docker performs the build stage.
- The application is compiled.
- Build artifacts are generated.

Step 6: Create the Runtime Image

- Docker copies the published application.
- Runtime dependencies are included.
- A final lightweight image is created.

Step 7: Verify the Image

- List all available Docker images.
- Execute this command.

docker images

```
powershell DockerApp X
PS D:\code\DockerApp> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
dockerapp:v1	eaf93774bebb	359MB	101MB	

```
PS D:\code\DockerApp>
```

Step 8: Run the Application Container

- Start a container from the image.

```
docker run -d -p 8080:80 --name dockerapp-container dockerapp:v1
```

```
PS D:\code\DockerApp> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
dockerapp:v1	eaf93774bebb	359MB	101MB	

```
PS D:\code\DockerApp> docker run -d -p 8080:80 --name dockerapp-container dockerapp:v1
4a140157e02e82a88037bde13b5979fa0d41d5aecb2f234d693b44fd75b38805
PS D:\code\DockerApp>
```

Step 9: Verify the Running Container

- Display the running containers.

```
docker ps
```

```
powershell DockerApp X
PS D:\code\DockerApp> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
dockerapp:v1	eaf93774bebb	359MB	101MB	

```
PS D:\code\DockerApp> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
4a140157e02e	dockerapp:v1	"dotnet DockerApp.dll"	About a minute ago	Up About a minute	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp

```
PS D:\code\DockerApp>
```

Step 10: Test the Application

- Open a web browser.
- Navigate to:
<http://localhost:8080>
- Confirm that the application is running successfully.

